

Лабораторная работа №12

Операционные системы

Кац Григорий Михайлович

Российский университет дружбы народов, Москва, Россия

7 октября 2026г.

Информация

Докладчик

- ▶ Кац Григорий Михайлович
- ▶ Студент НКАбд-03-25
- ▶ Российский университет дружбы народов
- ▶ 1032251955@rudn.ru

1. Цель работы

Освоить базовые принципы скриптовой разработки в среде UNIX/Linux и приобрести навыки создания простых сценариев командной оболочки.

2. Задание

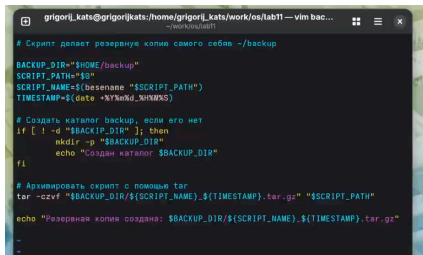
1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию `backur` в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор `zip`, `bzip2` или `tar`. Способ использования команд архивации необходимо узнать, изучив справку.
2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.
3. Написать командный файл — аналог команды `ls` (без использования самой этой команды и команды `dir`). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.
4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (`.txt`, `.doc`, `.jpg`, `.pdf` и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

3. Теоретическое введение

Командный интерпретатор (shell) представляет собой программу, обеспечивающую интерфейс взаимодействия пользователя с операционной системой. В среде UNIX/Linux распространены следующие разновидности оболочек: - Bourne shell (sh) — базовая стандартная оболочка с минимальным, но достаточным набором функций. - C shell (csh) — расширение sh с синтаксисом, напоминающим язык C, и поддержкой истории команд. - Korn shell (ksh) — гибрид, где управляющие операторы аналогичны sh, а остальные возможности близки к csh. - BASH (Bourne Again Shell) — разработка Free Software Foundation, объединяющая свойства csh и ksh. Отдельно выделяется стандарт POSIX (разработан IEEE), определяющий единые интерфейсы для совместимости UNIX-подобных ОС и переносимости приложений на уровне исходного кода. POSIX-совместимые оболочки базируются на оболочке Корна.

4. Выполнение лабораторной работы

1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.



```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab11 — vim bac...
~/.work/os/lab11

# Скрипт делает резервную копию самого себя в ~/backup

BACKUP_DIR="$HOME/backup"
SCRIPT_PATH="$0"
SCRIPT_NAME=$(basename "$SCRIPT_PATH")
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Создать каталог backup, если его нет
if [ ! -d "$BACKUP_DIR" ]; then
    mkdir -p "$BACKUP_DIR"
    echo "Создан каталог $BACKUP_DIR"
fi

# Архивировать скрипт с помощью tar
tar -czvf "$BACKUP_DIR/${SCRIPT_NAME}_${TIMESTAMP}.tar.gz" "$SCRIPT_PATH"

echo "Резервная копия создана: $BACKUP_DIR/${SCRIPT_NAME}_${TIMESTAMP}.tar.gz"

-
-
```

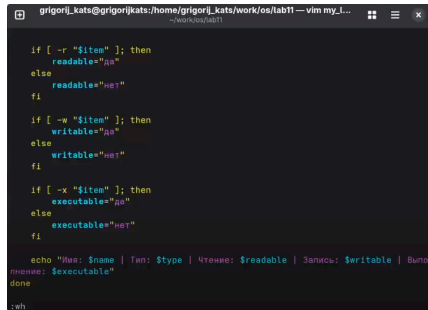
Рис. 1: Первый скрипт

2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.

```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab11 — vim print...  
~/.work/os/lab11  
#!/bin/bash  
# Обрабатывает любое число аргументов командной строки  
  
echo "Всего передано аргументов: $#"  
echo "Список всех аргументов: $@"  
echo "_____"  
  
i=1  
for arg in "$@"  
do  
    echo "Аргумент $i: $arg"  
    i=$((i+1))  
done
```

Рис. 2: Второй скрипт

3. Написать командный файл — аналог команды `ls` (без использования самой этой команды и команды `dir`). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.

A screenshot of a terminal window with a dark background. The window title is "grigorij_kats@grigorijkats: /home/grigorij_kats/work/os/lab11 — vim my_l...". The script content is as follows:

```
if [ -r "$item" ]; then
    readable="да"
else
    readable="нет"
fi

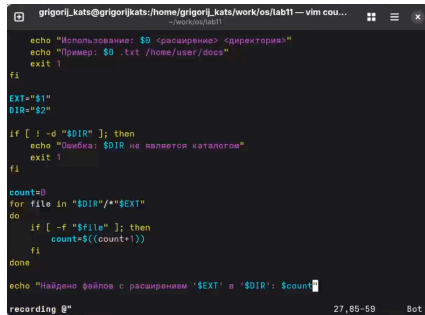
if [ -w "$item" ]; then
    writable="да"
else
    writable="нет"
fi

if [ -x "$item" ]; then
    executable="да"
else
    executable="нет"
fi

echo "Имя: $name | Тип: $type | Чтение: $readable | Запись: $writable | Выпo
ление: $executable"
done
:wh
```

Рис. 3: Третий скрипт

Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента ко- мандной строки.



```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab11 — vim cou...
~/work/os/lab11

echo "Использование: $0 <расширение> <директория>"
echo "Пример: $0 .txt /home/user/docs"
exit 1
fi

EXT="$1"
DIR="$2"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом"
    exit 1
fi

count=0
for file in "$DIR"/*"$EXT"
do
    if [ -f "$file" ]; then
        count=$((count+1))
    fi
done

echo "Найдено файлов с расширением '$EXT' в '$DIR': $count"

recording @ 27,85-59 Bot
```

Рис. 4: Четвертый скрипт

Делаю файлы исполняемыми и запускаю.

```
grigorij_kats@grigorijkats:~/work/os/lab11$ vi count_files.sh
grigorij_kats@grigorijkats:~/work/os/lab11$ chmod +x count_files.sh
grigorij_kats@grigorijkats:~/work/os/lab11$ mkdir -p ~/work/os/lab11/testdir
grigorij_kats@grigorijkats:~/work/os/lab11$ touch ~/work/os/lab11/testdir/a.txt
~/work/os/lab11/testdir/b.txt ~/work/os/lab11/testdir/c.jpg
grigorij_kats@grigorijkats:~/work/os/lab11$ ./count_files.sh .txt ~/work/os/lab11/testdir
```

Рис. 5: Запуск

5. Выводы

Нами освоены основы shell-программирования в UNIX/Linux и приобретены навыки написания простых скриптов.